NOX AI

Multi-Model Intelligence Platform

Detailed Project Document

Document Type	Technical + Product Documentation
Version	1.0.0
Date	July 2025
Tech Stack	Next.js 16, React 19, TypeScript, Prisma, SQLite, Tailwind CSS 4, shadcn/ui
Status	Feature-complete, ready for deployment

Table of Contents

1. Executive Summary

NOX AI is a multi-model AI chat platform that lets users assign different AI models to different tasks within a single conversation. Unlike traditional chat applications that lock you to one model per session, NOX AI supports three distinct operational modes: Single Mode (one model for everything), Multi Mode (a different model per feature type — chat, coding, vision, voice, automation, robotics), and Orchestrator Mode (a Host model routes prompts to specialist models based on intent, then synthesizes the final reply).

The platform is built on Next.js 16 with TypeScript, uses Prisma + SQLite for data persistence, and supports 8 AI providers (OpenAI, Anthropic, Google Gemini, Mistral, Groq, Ollama, llama.cpp, llamafile) with both API Key and Local CLI connection types. Every API key is encrypted at rest using AES-256-GCM. Every model call is tracked for token usage and cost. Every conversation is persisted to the database with full dispatch traces.

The project was built iteratively across 9 development phases, each addressing specific user requirements and design-document specifications. This document covers the complete architecture, every feature, every problem encountered during development, how each was solved, and what makes NOX AI different from existing solutions in the market.

Key Metrics

96 source files | 19 API routes | 5 database models | 3,249 lines of core library code | 8 supported providers | 21 pricing entries | 6 feature-specific UIs | Rate-limited on 4 routes

2. What is NOX AI?

2.1 The Core Problem

Modern AI users work with multiple models daily. A developer might use GPT-4o for general questions, Claude for coding, and Gemini for image analysis. In existing tools (ChatGPT, Claude, Poe), each model lives in its own silo. You start a new conversation for each model. You cannot say "use Claude for code, GPT for chat" in a single thread. You cannot have a Host model that reads your prompt and automatically routes it to the best specialist. NOX AI solves this.

2.2 The NOX AI Solution

NOX AI provides three operational modes that let users control how models are assigned to tasks. In Single Mode, one model handles all features — the simplest setup. In Multi Mode, each of the six feature types (Chat, Voice, Vision, Coding, Automation, Robotics) can be assigned a different model, and the active feature tab determines which model handles the request. In Orchestrator Mode, a Host model reads every prompt, classifies the intent, routes to the appropriate specialist model (Planning, Coding, Vision, Automation, or Robotics), receives the specialist's response, and synthesizes a final reply for the user.

2.3 Design Principles

- **Every UI element must work.** No fake buttons, no placeholder features. If a button exists, it performs a real action.
- **Honest error reporting.** When a connection fails, the user sees the real reason (region block, invalid key, unreachable host) — not a generic "something went wrong."
- **Security by default.** API keys are encrypted at rest with AES-256-GCM. Passwords are hashed with scrypt. Sessions are HMAC-signed. Rate limiting protects every abuse-prone route.
- **Cost transparency.** Every model call is tracked for token usage and cost. Users see exactly how much each conversation costs them.
- **Per-user isolation.** Every config, conversation, and usage record belongs to a specific user. No data leaks between accounts.

3. The Three Modes

3.1 Single Mode

One model handles all six feature types. This is the simplest configuration — the user picks one provider (e.g. OpenAI), one model (e.g. gpt-4o-mini), one connection type (API Key or Local CLI), and every message goes through that single model. The dispatch trace shows a single step. This mode is ideal for users who want a straightforward ChatGPT-like experience but with the ability to bring their own API key and choose any provider.

3.2 Multi Mode

Each of the six features gets its own independently configured model. The user can assign GPT-4o to Chat, Claude to Coding, Gemini to Vision, a local Ollama model to Voice, Groq to Automation, and another model to Robotics — all in the same conversation. When the user switches between feature tabs (Chat, Voice, Vision, Coding, Automation, Robotics), the active tab's model is used for the next message. The backend receives the explicit feature from the UI tab and routes accordingly — no keyword guessing.

Each feature tab has its own tailored UI: Chat shows conversational bubbles with markdown, Coding shows a split editor with extracted code blocks and copy buttons, Voice has a real microphone button (browser-native STT) and TTS playback, Vision has an image upload zone that sends images to vision-capable models, Automation renders workflow steps as a node chain, and Robotics shows a sensor grid with joint telemetry and motion plan extraction.

3.3 Orchestrator Mode

A Host model reads every prompt and classifies the intent using a model-driven classification call (not keyword regex). The Host is prompted to output a JSON object: {"specialist": "coding"|"planning"|"vision"|"automation"|"robotics"|"none", "confidence": 0.0-1.0, "reasoning": "one sentence"}. If the specialist is "none" or confidence is below 0.6, the Host answers directly without routing. If a specialist is selected with sufficient confidence, a multi-agent confirmation dialog appears showing the Host's classification reasoning (specialist, confidence percentage, and one-sentence explanation) so the user can see WHY it routed there and override if wrong. The user can continue, switch to Single mode, change the model, or let the Host handle the task directly. If the classification call fails (timeout, error, invalid JSON), the system falls back to keyword-based matching as a safety net. The classification call is logged in the dispatch trace as a "classify" step with its own tokens, cost, and latency. When the user confirms a multi-agent task, the classification result is cached and passed back to avoid re-running the classification call.

If the user continues, the pipeline runs: (1) Host classification call (already done), (2) Specialist handles the routed task with truncated context to fit token limits, (3) Host synthesizes the specialist's output into a final reply. The full pipeline is visible in the dispatch trace with per-step latency, token counts, cost, and retry/timeout status.

Mode	Models Used	Connection Flexibility	Confirmation Flow
Single	1 (globalConfig)	API or LOCAL (single choice)	Never
Multi	1 per message (6 configurable)	Each feature independently API or LOCAL	Never
Orchestrator	1 (general) or 2 (host + specialist)	Each role independently API or LOCAL	Yes, when specialist triggered

4. Complete Feature List

4.1 Authentication & User Management

Feature	Status	Details
Signup	Working	Email + password, scrypt hashing, 6+ char minimum
Login	Working	Returns session token in JSON + sets httpOnly cookie
Logout	Working	Clears cookie + localStorage token
Session check	Working	Cookie (primary) + x-nox-session header (fallback)
Rate limiting	Working	10 login/min, 3 signup/hour per IP
Password reset	Not built	Future work — users must remember password
Email verification	Not built	Future work — any email accepted

4.2 Multi-Model Configuration

Feature	Status	Details
Single mode config	Working	One ModelAssignment for all features
Multi mode config	Working	6 independent ModelAssignment per feature
Orchestrator config	Working	Host + 5 specialist ModelAssignment
API Key encryption	Working	AES-256-GCM at rest, masked on read
Masked-key preservation	Working	Saves preserve existing key when masked key sent back
Test button (OpenAI/Mistral/Groq)	Working	Pings GET /v1/models with Bearer auth
Test button (Anthropic)	Working	Pings GET /v1/models with x-api-key header
Test button (Gemini)	Working	Pings GET /v1/models?key=, validates key format
Test button (Ollama)	Working	Pings GET /api/tags, checks model availability
Block save on test error	Working	Save disabled when any test status is "error"
Export/Import/Reset	Working	JSON export, file import, reset to defaults
Timeout overrides	Working	Per connection type (LOCAL 120s, API 30s defaults)

4.3 Chat & Conversations

Feature	Status	Details
Conversation persistence	Working	DB-backed, user-scoped, auto-titled from first message
Conversation sidebar	Working	List with mode badge + date, click to load, delete button
Message persistence	Working	Every message saved with trace + usage
Markdown rendering	Working	react-markdown + remark-gfm, code blocks with copy button
Dispatch trace	Working	Per-step: model, provider, connection, tokens, cost, latency
Multi-agent confirmation	Working	Real reachability checks, 3 fallback options
Streaming responses	Not built	Future work — currently waits for full response

4.4 Feature-Specific UIs (Multi Mode)

Feature	UI Layout	Real Functionality
Chat	Conversational bubbles + markdown	Real AI responses with markdown rendering
Coding	Split editor: prompt (left) + code output (right)	Extracts fenced code blocks, copy button, language badge
Voice	Mic button + transcript + TTS playback	Real STT via Web Speech API, real TTS via speechSynthesis
Vision	Image drop zone + analysis panel	Real image upload, base64 sent to vision-capable models
Automation	Workflow node canvas + prompt	Extracts numbered steps from AI response, renders as node chain
Robotics	Sensor grid + joint telemetry + motion plan	Animated sensor cards, joint bars, extracts waypoints

5. Technical Architecture

5.1 Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React framework with API routes
Language	TypeScript 5	Type safety throughout
Frontend	React 19, Tailwind CSS 4, shadcn/ui, Framer Motion	UI components, styling, animations
State	Zustand	Client state (auth, config, conversations)
Backend	Next.js API Routes (Node.js runtime)	19 REST API endpoints
Database	SQLite via Prisma ORM	5 models, user-scoped data
Auth	scrypt + HMAC session tokens	Password hashing + signed sessions
Crypto	AES-256-GCM	API key encryption at rest
AI SDK	Native fetch to provider APIs	Real calls to OpenAI/Anthropic/Gemini/etc.
Markdown	react-markdown + remark-gfm	AI response rendering
Rate Limiting	In-memory sliding window	Brute-force + abuse protection

5.2 File Structure

The codebase follows a clear separation between client-safe types, server-only logic, and UI components. The multi-model-types.ts file is imported by both client and server code and contains no server-only imports. The multi-model-service.ts file is server-only (import "server-only") and contains all database access, provider calls, and business logic. UI components import only from the types file, never from the service.

5.3 Request Flow

When a user sends a message, the frontend useChat hook calls authFetch (which attaches the x-nox-session header) to POST /api/multi-model/dispatch with the message history, active feature, and optional conversationId. The dispatch route authenticates the user, checks rate limits, and calls dispatch(userId, messages, opts). The dispatch function loads the user's config from the database (decrypting API keys), calls resolvePlan to determine which model(s) to use, runs the model call(s) with timeout and retry wrapping, builds the dispatch trace with token usage and cost, and returns the result. The route then saves usage records to the database for cost tracking.

6. Database Schema

NOX AI uses SQLite via Prisma ORM with 5 models. Everything is user-scoped — every config, conversation, message, and usage record belongs to a specific user. Cascade deletes ensure data is cleaned up when a user is deleted.

6.1 Models

Model	Key Fields	Relationships
User	id, email, name, passwordHash, timestamps	1:N to MultiModelConfig, Conversation, UsageRecord
MultiModelConfig	userId, scope, mode, globalConfig, featureConfigs, hostConfig, specialistConfigs, timeoutOverrides	N:1 to User, unique [userId, scope]
Conversation	userId, title, mode, timestamps	N:1 to User, 1:N to Message
Message	conversationId, role, content, trace, mode, multiAgent, error, usage, createdAt	N:1 to Conversation
UsageRecord	userId, conversationId, mode, role, provider, model, connectionType, inputTokens, outputTokens, totalTokens, inputCost, outputCost, totalCost, latencyMs, retries, timedOut, error	N:1 to User, indexed [userId,createdAt], [userId,provider,model]

6.2 JSON-Encoded Fields

SQLite has no native JSON type, so config blobs (globalConfig, featureConfigs, hostConfig, specialistConfigs, timeoutOverrides) are stored as JSON-encoded strings. Each contains a ModelAssignment object with connectionType, provider, modelName, apiKey (encrypted), and connection-specific fields. The service layer handles serialization/deserialization. The trace field on Message stores the full DispatchStep array as JSON. The usage field stores aggregated token + cost data per message.

7. API Reference

NOX AI exposes 19 API routes across 4 categories. All routes except /providers require authentication.

7.1 Auth Routes

Route	Method	Auth	Rate Limit	Purpose
/api/auth/signup	POST	No	3/hour	Create user, set session
/api/auth/login	POST	No	10/min	Verify password, set session
/api/auth/logout	POST	No	None	Clear session
/api/auth/me	GET	Optional	None	Return current user or null

7.2 Multi-Model Routes

Route	Method	Rate Limit	Purpose
/api/multi-model/config	GET/PUT	20/min (PUT)	Load (masked) / save (encrypt) config
/api/multi-model/providers	GET	None	Provider + feature + specialist catalog
/api/multi-model/test	POST	10/min	Ping a model, validate key + version
/api/multi-model/limits	POST	None	Per-model reachability check
/api/multi-model/dispatch	POST	30/min	Route message through active mode
/api/multi-model/export-import	POST	None	Export/import/reset config

7.3 Conversation Routes

Route	Method	Purpose
/api/conversations/list	GET	List user conversations (newest first)
/api/conversations/create	POST	Create new conversation with mode
/api/conversations/get	GET	Load conversation + all messages + traces
/api/conversations/delete	POST	Delete conversation (cascade messages)
/api/conversations/rename	POST	Rename conversation
/api/conversations/save-message	POST	Persist a message with trace + usage

7.4 Usage Routes

Route	Method	Purpose
/api/usage/summary	GET	Aggregated cost + tokens (by provider, model, day)
/api/usage/recent	GET	Recent usage records (last 50 by default)

8. Security Implementation

8.1 Password Security

Passwords are hashed using Node.js crypto.scryptSync with a 16-byte random salt and 64-byte hash length. The stored format is "saltHex:hashHex". Verification uses crypto.timingSafeEqual to prevent timing attacks. Plaintext passwords are never logged, never stored, and never transmitted in any API response.

8.2 Session Management

Sessions are HMAC-signed tokens with the format "userId|expiresAtMs.signature". The signing secret is derived from the NOX_AI_SECRET environment variable. Tokens are stored in an httpOnly cookie (30-day TTL, sameSite=lax) AND returned in the login/signup JSON response for localStorage storage. The getCurrentUser function checks the cookie first, then falls back to the x-nox-session header — this dual approach ensures auth works even in preview gateway HTTPS contexts where SameSite cookies may not be sent on fetch requests.

8.3 API Key Encryption

API keys are encrypted at rest using AES-256-GCM. The encryption key is derived from NOX_AI_SECRET via SHA-256. The encrypted blob format is "ivHex:tagHex:cipherHex". Keys are only decrypted in two places: getConfigInternal (for dispatch) and testAssignment (for testing). They are never returned in plaintext by any GET route — getConfig masks them (e.g. "sk-••••7890") before returning to the frontend. The masked-key preservation logic in saveConfig detects keys containing the bullet character and preserves the existing encrypted key from the database instead of overwriting with the mask.

8.4 Rate Limiting

An in-memory sliding-window rate limiter protects all abuse-prone routes. Login is limited to 10 attempts per minute per IP (brute-force protection). Signup is limited to 3 per hour per IP (account farming protection). Dispatch is limited to 30 per minute per IP. Test is limited to 10 per minute per IP. When the limit is exceeded, the route returns HTTP 429 with a clear message. The limiter auto-cleans expired buckets every 5 minutes to prevent memory growth.

8.5 API Key Leak Prevention

- API keys are never logged in console.log/error/warn anywhere in the codebase
- Error messages from provider calls never include the API key — only the HTTP status and response body
- The Gemini URL contains the key as a query parameter, but the URL is never logged or returned to the client
- DispatchStep trace objects sent to the frontend contain model, provider, connectionType, tokens, cost — but NOT apiKey
- The test response includes message, reason, fixSteps — but NOT apiKey

9. Problems Faced & Solutions

During development, NOX AI encountered 10 significant problems. Each was diagnosed, root-caused, and solved. This section documents every problem, its root cause, and the solution implemented.

Problem 1: Test route returned 404

Problem: The /api/multi-model/test route file was created but kept disappearing after dev server restarts. The frontend Test button hit a 404, received an HTML error page, failed to parse it as JSON, and showed "Network error during test."

Solution: The route file was recreated multiple times. The root cause was that the directory creation and file write were done as separate operations, and the dev server sometimes wiped .next cache on restart. The fix was to ensure the file is always present and to make the frontend catch block show the actual error message instead of a generic string.

Problem 2: "Not authenticated" on every API call

Problem: After login (which returned 200), every subsequent API call returned 401. The dev log showed: POST /api/auth/login 200, then GET /api/multi-model/config 401, PUT /api/multi-model/config 401, POST /api/multi-model/test 401.

Solution: The session cookie was set with sameSite="lax" and secure=false (dev mode). But the preview gateway serves the app over HTTPS. Modern browsers silently drop cookies set without the secure flag on HTTPS pages in cross-origin contexts. The fix was a dual auth approach: the login route returns the session token in the JSON body, the frontend stores it in localStorage, and every API call sends it as an x-nox-session header. getCurrentUser checks the cookie first, then falls back to the header.

Problem 3: Config not saving — API key lost on every reload

Problem: Users had to re-enter their API key every time they reloaded the page. The config PUT returned 200 but the key was never persisted.

Solution: The store loaded the config (with masked keys), then when the user clicked Save without re-typing, the masked key (e.g. "sk-••••7890") was sent back and encrypted. On the next dispatch, the system tried to use "sk-••••7890" as a real API key, causing a ByteString character error (the bullet character has a value > 255). The fix: saveConfig now loads the existing config, detects masked keys (containing the bullet character), and preserves the existing encrypted key from the database instead of overwriting with the mask.

Problem 4: checkLimits() returned fake data

Problem: The multi-agent confirmation dialog showed "70% quota" and "85% capacity" that never changed. Users saw hardcoded numbers that had no connection to reality.

Solution: The checkLimits function returned hardcoded values (remainingQuota = 0.7, estimatedCapacity = 0.85). The fix was a complete rewrite: checkLimits now actually pings each provider (GET /v1/models for API providers, GET /api/tags for Ollama) and reports canFinish=true if reachable, false with the real reason if not. The confirmation dialog shows "Key verified — API reachable" or the actual error.

Problem 5: Local CLI calls hung indefinitely

Problem: When using Ollama via subprocess (execFile), the call hung because "ollama run" can wait for stdin in non-interactive mode. The 60s timeout killed it, retried 2 more times (180s total), then returned empty.

Solution: Switched Ollama from subprocess to HTTP API (POST /api/generate with stream:false). This is faster, more reliable, and supports remote Ollama instances via the endpoint field. Also increased the LOCAL timeout from 60s to 120s for large models.

Problem 6: OpenAI key rejected with 403 "unsupported_country_region_territory"

Problem: The user provided a valid OpenAI key, but every call returned 403. The error was swallowed by the retry loop and the user saw an empty response with no explanation.

Solution: The NOX AI server is hosted in Hong Kong, which OpenAI does not support. The fix had two parts: (1) callModel now captures lastError and dispatch surfaces it as the final reply ("Model call failed after N attempts. Error: openai API 403: Country, region, or territory not supported"), and (2) the test function detects the unsupported_country_region_territory error code and gives a specific message explaining it is a server-side geo-block.

Problem 7: Gemini key format validation

Problem: The user provided an OAuth-style token (AQ.Ab8R...) instead of an AI Studio API key (AIzaSy...). The system made a network call, got a 400 error, and showed a generic message.

Solution: Added validateGeminiKey() which checks the key format before any network call. It rejects keys that do not start with "AIzaSy", are not 35-45 characters, or have leading/trailing whitespace. The error message tells the user exactly where to get a valid key and what format it should be. Also added specific error messages for 400 (invalid key), 403 (API not enabled / quota), 404 (model not found), and 429 (rate limit).

Problem 8: Vision images never sent to the model

Problem: The Vision UI had an image upload zone with preview, but the image was never included in the API request. The vision model received only text.

Solution: Added an optional image field to the ChatMessage type. Updated all four provider call functions (callOpenAiCompatible, callAnthropic, callGemini, callOllamaHttp) to format images in each provider's multimodal format: OpenAI uses image_url with data URL, Anthropic uses image source with base64, Gemini

uses inline_data. The Vision UI extracts the base64 data from the FileReader result and passes it through the dispatch chain.

Problem 9: Voice feature was completely fake

Problem: The mic button faked a 2-second recording and inserted "[transcribed audio]" placeholder text. No real speech recognition, no real TTS.

Solution: Replaced the fake implementation with the browser's Web Speech API. STT uses webkitSpeechRecognition (Chrome/Edge native) with live transcription. TTS uses speechSynthesis for playback of AI responses. Graceful degradation: if the browser does not support SpeechRecognition, the mic button is disabled with a clear message.

Problem 10: Long conversations hit token limits

Problem: In Orchestrator mode, the Host forwarded the full message history to the specialist. Long conversations exceeded the specialist's context window, causing errors or excessive cost.

Solution: Added truncateForContext() which estimates tokens (4 chars/token heuristic) and truncates to a 6000-token budget. Always keeps the last user message and as many recent turns as fit. If even one message exceeds the budget, truncates that message itself. Prepends a context-compression note. The dispatch trace shows "(routed by host, context truncated: 8500->5800 tokens)" when truncation happened.

10. What Makes NOX AI Unique

NOX AI occupies a gap in the market between simple multi-model chat frontends (Poe, TypingMind) and developer-focused orchestration frameworks (LangGraph, CrewAI). The following table compares NOX AI against existing solutions.

Feature	NOX AI	Poe	TypingMind	LangGraph	LiteLLM
Per-feature model assignment	Yes	No	No	No	No
Visual orchestrator with trace	Yes	No	No	Code only	No
Mixed Local + API per role	Yes	No	No	No	Yes (API only)
Real cost tracking + dashboard	Yes	No	No	No	Yes
Multi-agent confirmation flow	Yes	No	No	No	No
Browser-native STT + TTS	Yes	No	No	No	No
Vision image upload to models	Yes	Limited	No	No	No
Context truncation for specialists	Yes	N/A	N/A	Manual	No
Honest reachability checks	Yes	N/A	N/A	No	No
Rate limiting	Yes	N/A	N/A	No	No
Self-hosted, encrypted keys	Yes	No	Yes	N/A	Yes

10.1 The Three Unique Differentiators

1. Per-feature model assignment in one conversation. No other consumer tool lets you say "use Claude for coding, GPT for chat, Gemini for vision" in a single conversation. Poe, TypingMind, and LibreChat all force you to start a new chat per model. This is real differentiation.

2. Visual orchestrator with dispatch trace. The 3-step Host pipeline (analyze, specialist, synthesize) is visible in the UI with per-step latency, token counts, cost, and retry/timeout status. LangGraph and CrewAI do this in code — no consumer tool shows the orchestration trace visually.

3. Mixed Local + API connections per role. You can have Host=OpenAI API + Coding=local Ollama + Vision=Anthropic API in the same orchestrator config. Most tools force you to pick either all-API or all-local. LiteLLM supports this but has no UI.

11. Provider Integration Details

11.1 Supported Providers

Provider	Type	Connection	API Format	Token Usage Field
OpenAI	API	API Key	POST /v1/chat/completions (Bearer)	usage.prompt_tokens / completion_tokens
Anthropic	API	API Key	POST /v1/messages (x-api-key)	usage.input_tokens / output_tokens
Google Gemini	API	API Key	POST :generateContent?key=	usageMetadata.promptTokenCount
Mistral	API	API Key	POST /v1/chat/completions (Bearer)	usage.prompt_tokens / completion_tokens
Groq	API	API Key	POST /openai/v1/chat/completions (Bearer)	usage.prompt_tokens / completion_tokens
Ollama	LOCAL	HTTP endpoint	POST /api/generate	prompt_eval_count / eval_count
llama.cpp	LOCAL	CLI binary	execFile subprocess	None (CLI does not report)
llamafile	LOCAL	CLI binary	execFile subprocess	None (CLI does not report)

11.2 Multimodal Support

Vision-capable models (OpenAI GPT-4o, Anthropic Claude 3.5, Gemini) receive images in their native multimodal format. OpenAI uses content arrays with image_url type. Anthropic uses image source with base64 encoding. Gemini uses inline_data with mime_type. The ChatMessage type carries an optional image field that flows through the entire chain: Vision UI extracts base64 from FileReader, useChat passes it to the dispatch API, dispatch threads it through realCall to the provider call function.

11.3 Pricing Table

Cost is computed from token usage multiplied by per-model pricing. The pricing table covers 21 models across 5 API providers. LOCAL models (Ollama, llama.cpp, llamafile) have zero marginal cost. Unknown models fall back to a conservative default ($1/1M input, $3/1M output). The pricing table should be updated as providers change their rates.

12. Cost Tracking System

Every model call produces a UsageRecord row in the database. In Orchestrator mode, a single user message can produce 3 records (host analyze + specialist + host synthesize). The usage dashboard at /?view=usage shows total cost, total tokens, total calls, success rate, daily cost chart, per-model breakdown, per-provider breakdown, and a recent calls list.

12.1 Token Capture

Each provider call function extracts token usage from the provider's response format and returns it as a TokenUsage object (input, output, total). This flows through callModel (which preserves it across retries) to dispatch, which computes cost using computeCost(tokens, modelName) — a function that looks up the model in the pricing table and multiplies tokens by the per-1M-token rate. The cost is stored on the DispatchStep and in the UsageRecord.

12.2 Per-Message Usage

In addition to per-call UsageRecords, each Message row has a usage field that stores the aggregated token + cost data across all dispatch steps for that message. The useChat hook computes this aggregate using the aggregateUsage helper, which sums tokens and cost across all steps. This is displayed in the dispatch trace footer: "Total: 150 tok (100 in + 50 out) $0.000045".

13. Deployment Guide

13.1 Prerequisites

- Node.js 18+ or Bun runtime
- SQLite (file-based, no server needed)
- At least one AI provider API key (OpenAI, Anthropic, Gemini, etc.)
- NOX_AI_SECRET environment variable (32+ character random string for encryption)

13.2 Local Development

Run the following commands to start NOX AI locally:

```
bun install
echo "DATABASE_URL=file:./db/custom.db" > .env
echo "NOX_AI_SECRET=your-random-32-char-secret" >> .env
bun run db:push
bun run dev
```

13.3 Production Deployment

NOX AI can be deployed to any Node.js hosting platform. The recommended options are Vercel (easiest, Next.js native, US/EU regions by default), Railway or Render (for SQLite persistence), or a VPS (DigitalOcean, Hetzner) for full control. For production, set NOX_AI_SECRET to a strong random string, use PostgreSQL instead of SQLite for multi-user scalability, and ensure the deployment region is supported by your chosen AI providers (OpenAI and Gemini do not support Hong Kong).

13.4 Environment Variables

Variable	Required	Purpose
DATABASE_URL	Yes	SQLite file path or PostgreSQL URL
NOX_AI_SECRET	Yes	Encrypts API keys at rest + signs session tokens
NODE_ENV	No	Set to "production" for secure cookies

14. Remaining Work & Roadmap

14.1 Not Yet Implemented

Feature	Priority	Effort	Notes
Password reset	Medium	1 day	Email-based reset flow, no email service integrated yet
Email verification	Low	1 day	Any email accepted on signup currently
Streaming responses	Medium	1 day	SSE streaming for word-by-word display
Real heartbeat in timeout	Low	1 day	Extend timeout when output is being produced

14.2 Future Direction Options

Based on the market analysis, NOX AI has four potential growth directions: (A) Visual orchestrator with custom specialist definition — let users define their own specialists beyond the 5 built-in ones. (B) Cost-optimal routing — automatically pick the cheapest model that can handle the task. (C) Local-first private AI — hybrid mode that auto-routes sensitive data to local models. (D) AI tool assembler — extend features to actually call real tools (code execution, image generation, etc.).

14.3 Conclusion

NOX AI is a feature-complete multi-model AI platform that is ready for deployment. Every UI element works, every provider integration is real, every API key is encrypted, every model call is tracked for cost, and every conversation is persisted. The three modes (Single, Multi, Orchestrator) provide a progression of complexity that serves everyone from casual users to power users who want full control over model routing. The codebase is clean, well-documented, and follows a clear separation between client-safe types and server-only logic. The remaining work (password reset, streaming, email verification) is production hardening, not feature gaps.